

PARCIAL 2

Proyecto de Retrieval-Augmented Generation (RAG)

HE2: Inteligencia Artificial Aplicada a la Economía
Universidad de los Andes
2026-1

Modalidad: Proyecto grupal + Reporte en PDF + Video de sustentación en YouTube

Entregables: Notebook + Reporte PDF con hallazgos + Video (máx. 5 min)

Participación: TODOS los integrantes deben participar en el video

1. Descripción General

El Parcial 2 consiste en desarrollar un sistema completo de Retrieval-Augmented Generation (RAG) aplicado a un dominio específico. Los estudiantes deben identificar un problema real donde un LLM por sí solo no sea suficiente (por falta de contexto, información desactualizada o riesgo de alucinaciones), construir un pipeline RAG funcional que resuelva ese problema, y documentar todo el proceso en un notebook, un reporte en PDF y un video de sustentación.

El proyecto debe demostrar comprensión profunda de cada componente del pipeline RAG: desde la recolección y procesamiento de documentos, pasando por la generación de embeddings y almacenamiento vectorial, hasta la recuperación de fragmentos relevantes y la generación de respuestas contextualizadas. No se evalúa solo que el sistema funcione, sino que el equipo entienda por qué funciona y pueda justificar cada decisión técnica.

Se espera que cada grupo trabaje con documentos reales y auténticos relacionados con el dominio escogido. El sistema debe ser capaz de responder al menos 5 preguntas representativas, y los estudiantes deben evaluar la calidad de las respuestas de forma crítica.

2. Entregables

2.1 Notebook (Jupyter / Google Colab)

El notebook debe contener todo el desarrollo del proyecto, organizado de forma clara:

- Código documentado y comentado en cada celda relevante.
- Celdas de Markdown explicando cada etapa del pipeline RAG.
- Visualizaciones relevantes (e.g., distribución de tamaños de chunks, scores de similitud, comparaciones de respuestas).
- Resultados de las consultas de prueba claramente presentados.

2.2 Reporte en PDF

Deben entregar un reporte formal en formato PDF que documente todos los hallazgos del proyecto. El reporte debe incluir:

- Portada: Título del proyecto, nombres de los integrantes, fecha.
- Introducción: Contexto del problema, por qué un RAG es la solución adecuada (vs. fine-tuning o prompting directo), objetivo del sistema.
- Dataset y documentos: Fuentes utilizadas, descripción de los documentos, justificación de la selección.
- Arquitectura del sistema: Descripción del pipeline completo, modelos seleccionados (encoder, decoder, reranker si aplica), base de datos vectorial.
- Resultados: Preguntas de prueba, respuestas generadas, análisis de calidad.
- Limitaciones y conclusiones: Problemas encontrados, propuestas de mejora, lecciones aprendidas.
- Bibliografía: Fuentes de los documentos utilizados y referencias técnicas.

El reporte debe ser conciso pero completo. No es una copia del notebook; es un documento ejecutivo que cualquier persona pueda leer y entender sin necesidad de ver el código.

2.3 Video de Sustentación (YouTube)

Deben grabar y subir un video a YouTube (puede ser no listado) con las siguientes características:

- Duración máxima: 5 minutos. Videos que excedan este límite serán penalizados.
- Participación obligatoria: TODOS los integrantes del grupo deben hablar y explicar al menos una parte del proyecto.
- Contenido: El video debe cubrir las secciones principales del proyecto (problema, documentos, arquitectura RAG, demo en vivo, resultados).
- Formato sugerido: Compartir pantalla mostrando el notebook mientras explican. Incluir una demo en vivo del sistema respondiendo al menos 2 preguntas.

IMPORTANTE: Integrantes que no participen en el video recibirán una nota de 0.0 en el parcial. No se aceptan excusas posteriores. Planifiquen con antelación.

3. Estructura del Proyecto

El proyecto debe cubrir las siguientes etapas de forma completa:

3.1 Planteamiento del Problema y Justificación del RAG

1. Definir claramente el problema que se busca resolver y el dominio de aplicación.
2. Explicar por qué un LLM genérico no es suficiente para resolver este problema (falta de contexto específico, información desactualizada, riesgo de alucinaciones).
3. Justificar por qué RAG es más adecuado que fine-tuning para este caso particular.
4. Identificar los usuarios objetivo del sistema y cómo se beneficiarían.
5. Relacionar el problema con principios económicos o del dominio correspondiente.

3.2 Dataset y Recolección de Documentos

1. Seleccionar un mínimo de 8 documentos reales y auténticos relacionados con el dominio (PDFs, artículos, manuales, reportes, normativas, etc.).
2. Describir cada documento: fuente, tipo de contenido, relevancia para el sistema.
3. Explicar el proceso de recolección y cualquier criterio de selección aplicado.
4. Los documentos deben estar disponibles en un repositorio de GitHub para verificación.

3.3 Procesamiento de Documentos (Chunking)

1. Extraer el texto de los documentos utilizando librerías adecuadas (e.g., pypdf, pdfplumber).
2. Implementar una estrategia de chunking: definir tamaño del fragmento y superposición (overlap). Justificar los valores elegidos.

3. Reportar estadísticas básicas: número total de fragmentos generados, distribución de tamaños, ejemplo de fragmentos resultantes.
4. Discutir: ¿qué pasa si los chunks son muy grandes o muy pequeños? ¿Cuál es el trade-off?

3.4 Embeddings y Base de Datos Vectorial

1. Seleccionar un modelo de embeddings (encoder) y justificar la elección. Explicar qué tipo de modelo es y por qué sirve para esta tarea.
2. Transformar cada fragmento en un vector (embedding) usando el modelo seleccionado.
3. Almacenar los vectores en una base de datos vectorial (e.g., FAISS, Chroma, Pinecone, Weaviate). Justificar la elección.
4. Explicar cómo funciona la búsqueda por similitud en la base de datos vectorial (e.g., similitud coseno, producto punto).
5. Reportar la dimensionalidad de los embeddings y la capacidad del modelo (tokens máximos por entrada).

3.5 Recuperación y Generación

1. Implementar el pipeline de recuperación: dado un query del usuario, convertirlo a embedding, buscar los top-K fragmentos más relevantes.
2. (Opcional, pero valorado) Implementar un reranker para mejorar la precisión de los fragmentos recuperados.
3. Seleccionar un modelo generador (decoder/LLM) y justificar la elección. Puede ser open-source (e.g., Mistral, LLaMA) o API (e.g., OpenAI, Anthropic).
4. Diseñar el prompt que combina la pregunta del usuario con el contexto recuperado. Explicar las instrucciones dadas al modelo.
5. Elegir el valor de K (número de fragmentos recuperados) y justificar. Discutir: ¿qué pasa si K es muy grande o muy pequeño?

3.6 Evaluación y Resultados

Formular al menos 5 preguntas representativas que un usuario real haría al sistema. Para cada pregunta:

- Mostrar los fragmentos recuperados y sus scores de similitud.
- Mostrar la respuesta generada por el sistema.
- Evaluar cualitativamente: ¿la respuesta es correcta? ¿Es completa? ¿El modelo alucinó?
- Incluir al menos un caso donde el sistema falle o responda de forma incompleta, y analizar por qué.
- Discutir si el sistema cita correctamente las fuentes y si las respuestas son trazables a los documentos originales.

4. Preguntas Guía para la Sustentación

Las siguientes preguntas son una guía de los temas que deben dominar. El video debe demostrar comprensión de estos conceptos. No memoricen respuestas; es más importante entender y saber explicar con claridad.

Sobre el Problema y la Justificación

- ¿Cuál es el problema que resuelve su sistema RAG?
- ¿Por qué un LLM genérico no puede resolver este problema por sí solo?
- ¿Cuál es la diferencia entre RAG y fine-tuning? ¿Por qué eligieron RAG?
- ¿Qué tipos de modelos interactúan en un RAG? ¿Cuál es encoder y cuál es decoder?
- ¿Qué problema resuelve RAG respecto a las alucinaciones de los LLMs?

Sobre los Documentos y el Procesamiento

- ¿Qué documentos seleccionaron y por qué?
- ¿Qué es chunking y por qué es necesario? ¿Qué pasa si no se hace?
- ¿Qué tamaño de chunk usaron y por qué? ¿Qué overlap usaron?
- ¿Qué problemas encontraron al extraer texto de los documentos?
- ¿Cuántos fragmentos generaron en total? ¿Es un número razonable?

Sobre los Embeddings y la Base de Datos Vectorial

- ¿Qué es un embedding? ¿Por qué es útil representar texto como vectores?
- ¿Qué modelo de embeddings usaron? ¿Es un encoder o un decoder? ¿Por qué?
- ¿Qué es una base de datos vectorial y en qué se diferencia de una base de datos tradicional?
- ¿Cómo funciona la búsqueda por similitud coseno? ¿Qué significa que dos vectores sean similares?
- ¿Qué limitación de capacidad (tokens) tiene su modelo de embeddings? ¿Cómo la resolvieron?

Sobre la Recuperación y la Generación

- ¿Qué valor de K usaron? ¿Por qué?
- ¿Qué es un reranker y para qué sirve? ¿Lo implementaron?
- ¿Qué modelo generador (decoder) usaron? ¿Por qué ese y no otro?
- ¿Cómo diseñaron el prompt? ¿Qué instrucciones le dieron al modelo?
- ¿El modelo alucinó en alguna respuesta? ¿Cómo lo identificaron?
- ¿Qué pasaría si el contexto recuperado no contiene la respuesta? ¿Cómo maneja eso su sistema?

Sobre los Resultados y Limitaciones

- ¿Las respuestas del sistema son correctas y útiles? ¿Cómo lo evaluaron?
- ¿En qué casos falla el sistema? ¿Por qué?
- ¿Cómo podrían mejorar el sistema?
- ¿El sistema es trazable? ¿El usuario puede verificar de dónde viene la información?

5. Rúbrica de Evaluación

Criterio	Peso	Descripción
Planteamiento del Problema y Justificación del RAG	10%	Problema claro, relevante, bien contextualizado. Justificación sólida de por qué RAG y no fine-tuning. Usuarios objetivo identificados.
Dataset y Procesamiento de Documentos	15%	Mínimo 8 documentos reales y relevantes. Extracción de texto adecuada. Estrategia de chunking justificada con parámetros razonables. Estadísticas reportadas.
Embeddings y Base de Datos Vectorial	15%	Modelo de embeddings justificado. Base de datos vectorial implementada y justificada. Comprensión de similitud vectorial demostrada. Capacidades y limitaciones del encoder reportadas.
Recuperación y Generación	20%	Pipeline de recuperación funcional. Modelo generador justificado. Prompt bien diseñado con instrucciones claras. Valor de K justificado. Reranker implementado (opcional pero bonificado).
Evaluación y Resultados	15%	Mínimo 5 preguntas probadas. Análisis cualitativo de cada respuesta. Caso de fallo incluido y analizado. Trazabilidad de fuentes evaluada.
Reporte en PDF	10%	Documento bien estructurado con todas las secciones requeridas. Redacción clara. Legible sin necesidad de ver el código.
Video de Sustentación	10%	Claridad en la explicación. Todos los integrantes participan. Demo en vivo del sistema. Dentro del límite de 5 minutos.
Calidad del Código y Notebook	5%	Código limpio, documentado, reproducible. Uso adecuado de Markdown. Organización clara del pipeline.

6. Reglas y Consideraciones

- **Grupos:** El trabajo se realiza en los grupos ya conformados para el curso.
- **Documentos:** Deben utilizar documentos reales y auténticos. No se permite inventar documentos o usar texto generado por IA como fuente del RAG.
- **Modelos:** Pueden usar modelos open-source (Hugging Face, Ollama) o APIs comerciales (OpenAI, Anthropic). Si usan APIs con costo, es responsabilidad del grupo.
- **Plagio:** Se verificará originalidad del código y las explicaciones. Copiar de otros grupos o de internet sin adaptación constituye plagio y se sanciona con nota de 0.0.

- **Herramientas permitidas:** Python (LangChain, LlamaIndex, Hugging Face, FAISS, ChromaDB, pypdf, sentence-transformers, etc.). Pueden usar Google Colab o Jupyter Notebook local.
- **Uso de IA generativa:** Pueden usar herramientas de IA como apoyo para el código, pero deben entender y poder explicar cada componente del pipeline. Si no pueden explicar algo en el video, se asumirá que no lo comprenden.
- **Límite del video:** Máximo 5 minutos. Por cada minuto adicional se descontarán 5 puntos sobre la nota del video.
- **GitHub:** Todo el proyecto (notebook, documentos fuente, reporte) debe estar en un repositorio de GitHub del grupo.

RECORDATORIO FINAL:

No memoricen respuestas. Es más importante comprender los conceptos y saber explicarlos con claridad. Relacionen siempre teoría y práctica en sus respuestas. Planifiquen el video con anticipación para que todos participen de forma equitativa.